

グラフ・ネットワークプログラム

釧路工業高等専門学校情報工学科5年・情報工学実験II

氏名：クリスティアン・ハルジュノ

出席番号：21

学籍番号：234071

提出日：2025年9月9日

目次

1	初めに	1
2	背景	1
2.1	グラフ探索アルゴリズム	1
2.1.1	課題 1: 深さ優先探索 (Depth-First Search, DFS)	1
2.1.2	課題 2: 幅優先探索 (Breadth-First Search, BFS)	2
2.2	全域木 (Spanning Tree)	3
2.3	課題 3: 連結成分数	4
2.4	課題 4: 最大クリーク	5
2.4.1	クリークの定義	5
2.4.2	クリーク問題の計算複雑性	6
3	実験方法	6
3.1	ハードウェアと環境	6
3.2	実験対象データ	7
3.2.1	課題 1 と課題 2: DFS と BFS の実装	7
3.2.2	課題 3: 連結成分数	8
3.2.3	課題 4: 最大クリーク	8
3.2.4	データの整形とパース	8
3.3	スタックとキューの実装	9
3.3.1	スタック	10
3.3.2	キュー	11
3.4	課題 1 と課題 2: DFS と BFS の実装	11
3.4.1	DFS の実装	11
3.4.2	BFS の実装	12
3.5	課題 3: 連結成分数	13
3.6	課題 4: 最大クリーク	14
3.6.1	最大クリーク探索には	14
4	実験結果	14
5	分析と理論	14
6	まとめ	14
7	発表についての感想	14
8	付録	14

1 初めに

グラフネットワーク理論は、対象間の関係をモデル化するために数学的構造を解析する研究である。専門的で限られた分野のように見えるが、グラフネットワーク理論は現代の計算機科学において大きな役割を果たしている。特に、地域間のインターネット通信を最適化する上で重要である。インターネットの基盤となるサーバーファームは世界中に点在している。最適化が行われなければ、自宅からこれらのサーバーへアクセスするには物理的距離の大きさにより多大な時間を要することになる。グラフ理論を用いることで、情報が通過する最適経路を見出し、インターネットアクセスの高速化を実現することが可能となる。

本実験では、ネットワーク理論の基礎を検討し、グラフネットワーク内でノードを探索する複数の手法を比較する。また、大規模なノードグラフに存在するクリークの数进行推定するために、グラフ計算アルゴリズムを最適化する手法についても検討する。

2 背景

2.1 グラフ探索アルゴリズム

グラフデータ構造を探索するアルゴリズムはいくつか存在する。本実験では、代表的なグラフ探索アルゴリズムの中でも特に広く用いられている深さ優先探索 (Depth-First Search, DFS) と幅優先探索 (Breadth-First Search, BFS) の二つを用いてグラフを探索する。

2.1.1 課題1: 深さ優先探索 (Depth-First Search, DFS)

DFSは木あるいはグラフのデータ構造を走査または探索するためのアルゴリズムである。このアルゴリズムは根から探索を開始し、ある枝を可能な限り深く進んだ後に親ノードへ戻り、他の枝を探索するという手順で動作する。DFSの概念自体は古くから知られ研究されてきたが、Tarjan (1972) がこのアルゴリズムを形式化し、さまざまな問題への応用を示した [7]。

DFSは幅よりも深さを優先する探索手法である。最も基本的な形態 (古典的反復DFS) では、探索対象のノードを保持するためにスタック (後入れ先出し, LIFO) のデータ構造を用いる。アルゴリズムはまず根ノードから探索を開始する。このノードをスタックに積み込み、取り出した後に訪問済みとして記録する。その後、このノードの子ノードをすべてスタックに積み込み、スタックが空になるまで同じ処理

を繰り返す [7]. 深さ優先探索アルゴリズムの可視化は図 1 に示される.

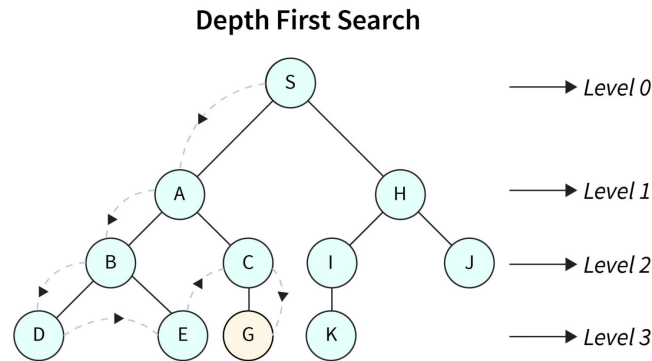


図 1: 深さ優先探索の手順 (DFS) の例

しかし, StackOverflow のコミュニティ投稿によれば, ノードをスタックから取り出した後に訪問済みとする方式では, グラフがサイクルを含む場合に同一ノードが複数回スタックに積まれる危険がある. この問題を回避するため, 本研究では修正版である反復 DFS (Modified Iterative DFS) を用いる. この手法ではノードをスタックに積み込む前に訪問済みとして記録する. これにより, ノードが再度スタックに積み込まれることを防ぐことができる [5].

2.1.2 課題 2: 幅優先探索 (Breadth-First Search, BFS)

深さ優先探索 (DFS) と同様に, 幅優先探索 (Breadth-First Search, BFS) アルゴリズムもグラフ・ネットワーク構造の走査あるいは探索に用いられる. DFS が一つの枝を可能な限り深く探索してから次の枝に移るのに対し, BFS はまずノードの子をすべて探索し, その後で次の階層へ進む. DFS がスタック (後入れ先出し, LIFO) を用いるのに対し, BFS はキュー (先入れ先出し, FIFO) を用いて探索対象のノードを管理する [6].

BFS も DFS と同様に根ノードから探索を開始する. 根ノードのすべての子ノード (根から直接導かれるノード) を訪問済みとして記録し, それらをキューに入れる. 次にキューの先頭からノードを取り出し, その子ノードを探索して訪問済みと

記録し, キューへ追加する. この処理をキューが空になり探索すべきノードがなくなるまで繰り返す [6]. 幅優先探索アルゴリズムの可視化は図 2 に示される.

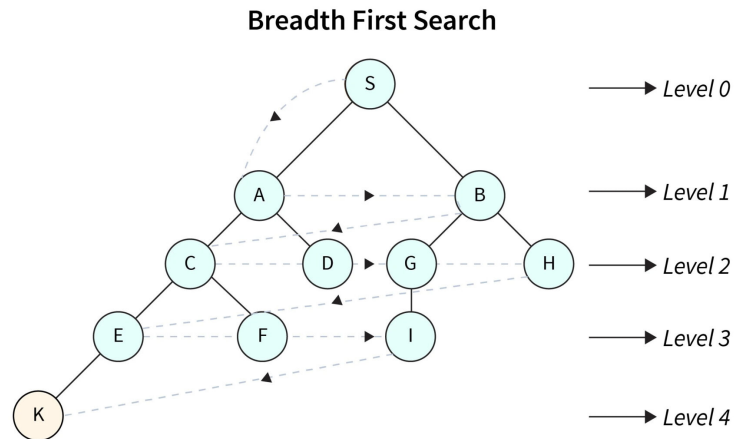


図 2: 幅優先探索の手順 (BFS) の例

元の DFS アルゴリズムを修正してサイクルを含むグラフでノードを再訪問しないようにしたのとは対照的に, BFS アルゴリズムはそのままの形でノードの再訪問を回避できる. これは, ノードをキューに追加する前に訪問済みとして記録するためである. この仕組みにより, 各ノードは一度しかキューに入らず, サイクルを含むグラフにおいても無限ループを防ぐことができる [6].

2.2 全域木 (Spanning Tree)

グラフ探索中に訪問したノードを記録すると, 得られる構造は全域木と呼ばれる. 全域木とは, 元のグラフ内の全ての頂点を含む部分グラフであり, 接続されておりサイクルを含まない木であり, 頂点数よりちょうど 1 つ少ない辺を持つ [9]. 隣接グラフから全域木への変換の例は, 図 3 に示されている.

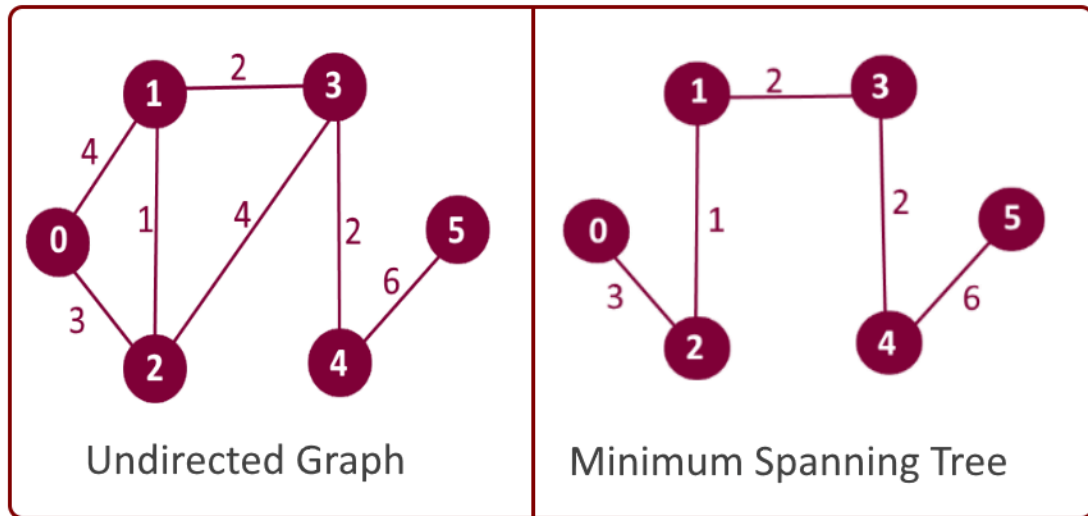


図 3: 隣接グラフから全域木への変換の例

グラフは探索の方法によって複数の全域木を持つことがある。DFS や BFS のような異なる探索方法は、異なる特徴を持つ全域木を生成する。DFS はできるだけ深く探索してからバックトラックするため、得られる全域木は非常に高さがあり細長くなることがある。一方、BFS は次のレベルに移動する前に全ての子ノードを探索するため、得られる全域木は低く広い形になる [9]。

2.3 課題 3: 連結成分数

一般に、グラフは大きく 2 種類に分類される: 連結グラフと非連結グラフである。連結グラフとは、任意の 2 頂点間に経路が存在するグラフを指す。この場合、任意の頂点から出発して全ての頂点に到達することが可能である [2]。一方で、非連結グラフとは、少なくとも 2 頂点間に経路が存在しないグラフを指す。この場合、始点から探索しても到達できない頂点が存在する [9]。このような状況により、グラフ内部には「連結成分」が形成される。非連結グラフは複数の連結成分を持つ場合がある。連結成分の個数は「連結成分数」と呼ばれる。例えば、図 4 では 2 つの連結成分が存在し、連結成分数は 2 となる。

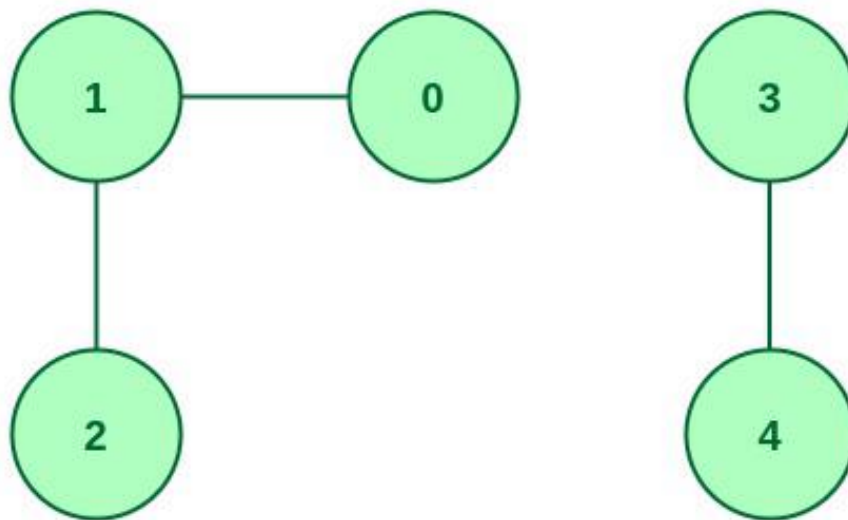


図 4: 2つの連結成分を持つ非連結グラフ

グラフにおける連結成分数を求める方法には、Union-Find、隣接行列、スペクトルグラフ理論など複数の手法が存在するが、最も容易で実装しやすい方法はDFSやBFSといったグラフ探索アルゴリズムを用いる方法である。連結成分数を計算する際には、グラフ内の全ての頂点を順に確認し、未訪問であれば探索を行う。探索に用いるアルゴリズムはDFSでもBFSでも問題なく、いずれも最終的には全ての頂点を訪問する。前章2.1で述べたように、実装されるアルゴリズムは訪問後に頂点を「訪問済み」として記録する。探索を実行するたびに連結成分数を1増加させる。探索アルゴリズムは単一の連結成分内の全ての頂点を訪問するため、最初の探索終了後にさらに探索を実行する場合、そのグラフは非連結であり、新たな連結成分として数えられる。この処理を全ての頂点が訪問済みになるまで繰り返す。最終的な連結成分数は探索を実行した回数に等しい [1]。

2.4 課題4：最大クリーク

2.4.1 クリークの定義

クリークとは、無向グラフ内の頂点の部分集合で、集合内の全ての頂点が互いに直接接続されているものを指す。言い換えると、クリークとはより大きなグラフの一部である完全グラフである [2]。サイズ k のクリーク (k -クリークとも呼ばれる) は、部分集合内の全ての頂点の組が辺によって接続されている k 個の頂点の部分集合である。グラフ内で可能な最大のクリークは最大クリークと呼ばれ、そのサイズはグラフ G のクリーク数 $\omega(G)$ と呼ばれる。

例えば、図5では、各グラフが異なるサイズのクリークを表している。各グラフはそれぞれ 3-クリーク、4-クリーク、5-クリークである [8]。各グラフは完全グラフであり、そのグラフ内の全てのノードが互いに接続されている。本実験では、与えられたグラフ内で最大または最も大きいクリークを見つけることが課題である。

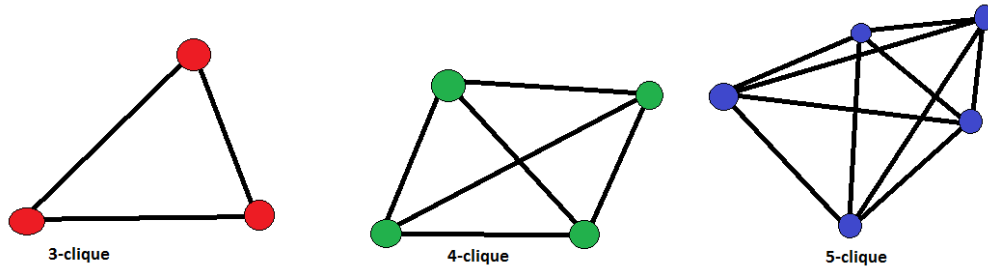


図 5: クリークの例

2.4.2 クリーク問題の計算複雑性

クリーク問題は計算複雑性理論における NP-完全問題として知られている [3]. NP-完全問題は、提案された解が多項式時間 ($O(n^k)$, 定数 k に対して) で検証可能な難しい問題のクラスである。問題が以下の条件を満たす場合、NP-完全と定義される [1]:

- 答えが「はい」または「いいえ」である決定問題である。
- 「はい」の解は多項式時間（短時間）で検証可能である。
- 「はい」の解の正確性は多項式時間で確認可能である。
- 問題は他の任意の NP 問題をシミュレートまたは表現可能であり、多項式時間で検証可能である。

3 実験方法

3.1 ハードウェアと環境

ハードウェアとしては Apple シリコンのプロセッサに基づいて実験を行う。

機種名 Apple Macbook Pro (M3, 2023 年モデル)

プロセッサ Apple M3 チップ (8 コア: 高性能コア 4 + 高効率コア 4)

GPU 10 コア統合型 GPU

メモリ 16GB ユニファイドメモリ

ストレージ 1TB SSD

OS macOS Sequoia 15.5

アーキテクチャ ARM64 (Apple シリコン)

また, 作成した実験プログラムは C 言語で書き, GCC コンパイラでコンパイルされる.

```
Apple clang version 17.0.0 (clang-1700.0.13.5)
```

```
Target: arm64-apple-darwin24.5.0
```

```
Thread model: posix
```

コンパイルを効率化するため, GCC コンパイラを Make と同時に利用する.

```
GNU Make 3.81
```

```
Copyright (C) 2006 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions.
```

```
There is NO warranty; not even for MERCHANTABILITY or
```

```
FITNESS FOR A PARTICULAR PURPOSE.
```

```
This program built for i386-apple-darwin11.3.0
```

3.2 実験対象データ

本研究で行う全ての実験は、隣接リストで表現された無向グラフを利用する. 各実験では、ノード数が異なる様々なサイズのグラフが与えられる. 実験で利用されるグラフは以下の通りである.

3.2.1 課題 1 と課題 2: DFS と BFS の実装

DFS と BFS の実装は、ノード数と密度が異なる 6 種類の無向グラフに対してテストされる. これらのグラフは隣接行列で表現され、密度に基づいて 2 種類に分類される. 密グラフはノード数に対して辺の数が多く、疎グラフは相対的に辺の数が少ない. 実験で利用されるグラフの詳細は表 1 にまとめられている.

表 1: 隣接行列の説明

ファイル名	ノード数	グラフの種類
search_100d.dat	100	密グラフ
search_100s.dat	100	疎グラフ
search_500d.dat	500	密グラフ
search_500s.dat	500	疎グラフ
search_1000d.dat	1000	密グラフ
search_1000s.dat	1000	疎グラフ

3.2.2 課題 3: 連結成分数

連結成分数の実験では、隣接行列で表現された 1 つの無向グラフを使用する。このグラフは 2000 を超えるノードを含み、辺と連結成分の数は未知である。グラフは search_2000.dat というファイルで与えられている。

3.2.3 課題 4: 最大クリーク

最大クリーク探索アルゴリズムは、隣接行列で表現された 5 種類の無向グラフに対してテストされる。グラフはサイズや密度が異なり、ノード数は 30 から 300 までの範囲にわたる。実験で使用されるグラフの詳細は表 2 にまとめられている。

表 2: クリーク探索に使用されるグラフの説明

ファイル名	ノード数	グラフの種類
clique_30.dat	30	密グラフ
clique_50.dat	50	密グラフ
clique_100.dat	100	密グラフ
clique_200.dat	200	密グラフ
clique_300.dat	300	密グラフ

3.2.4 データの整形とパース

本実験で使用される全てのグラフは、隣接行列として整形されたテキストファイルで与えられる。例えばグラフが 50 ノードを含む場合、隣接行列は 50×50 の行列として表現される。これは探索において小さな課題をもたらす。あるノードが他のノードに接続されているかを確認するためには、全てのノードを反復処理して接続の有無を確認する必要がある。その結果、計算量は $O(n^2)$ となり、大規模なグラフに対しては非効率になる可能性がある。探索処理を最適化するために、各

隣接行列は事前に処理され、各ノードがポインタを通じて他のノードにリンク可能な構造体として表現される隣接リストに変換される。

本実験で使用する各グラフノードの構造体はコード 1 に示されている。

コード 1: グラフノードの構造

```
struct Node{
    int id;
    bool visited;
    int connected_nodes_count;
    Node **connected_nodes;
};
```

現在のノードを追跡するために、id が置かれ、ノードが訪問済みかどうかを示すブール値も記録されている。接続されているノードの数も構造体に保存されており、接続ノードを反復処理する際に効率的である。最後に、他のノードへの接続を表現するために、ノードへのポインタ配列へのポインタが使用されている。これにより、全てのノードを反復処理する必要なく、接続されているノードへ直接アクセスすることが可能となり、グラフ探索を効率的に行うことができる。

探索の結果として得られる全域木も、ポインタによってリンクされたノードとして表現される。コード 1 と同様に、全域木の各ノードは id と隣接ノードへのポインタのリストを含む。唯一の違いは、親ノードへのポインタが存在する点である。全域木ノードの完全な構造はコード 2 に示されている。

コード 2: 全域木ノードの構造

```
struct TreeNode{
    int id;
    int connected_nodes_count;
    TreeNode **connected_nodes;
    TreeNode *parent_node;
};
```

3.3 スタックとキューの実装

DFS と BFS のアルゴリズムを実装するためには、スタックとキューを使用する必要がある [1]。標準の C 言語ライブラリにはスタックやキューの組み込みデータ構造が存在しないため、本研究ではこれらのデータ構造をリンクリストを用いて一から実装する。

C 言語における標準配列は固定サイズであり、動的にサイズを変更することができない。この制約により、要素の追加や削除に応じて動的に拡張や縮小を行うスタックやキューには不向きである。一方、リンクリストは動的にメモリを割り当てることができ、必要に応じてサイズを拡張または縮小することが可能である。配列は連続したメモリ領域に割り当てられるが、リンクリストのノードはメモリ上で分散していても、ポインタによって順序構造を維持できる。各リンクリストノードはデータと次のノードへのポインタを含む [1][4]。

3.3.1 スタック

スタックは本質的に LIFO (Last In First Out) 型のデータ構造である。スタックに最後に追加された要素が最初に取り出される。章 3.3 で述べたように、本実験におけるスタックは単方向リンクリストを用いて実装される。スタックの構造体はコード 3 に示されている。

コード 3: スタックの構造

```
struct StackNode{
    Node *graph_node; // グラフノードへのポインタ
    TreeNode *tree_parent; // 全域木ノードへのポインタ
    StackNode *next;
};
struct Stack{
    StackNode *head;
    int len;
};
```

スタックは 2 層構造を用いて実装される。1 つ目の構造体 `StackNode` はスタック内の各ノードを表す。スタックの長さとスタックの先頭を管理するために、2 つ目の構造体である `Stack` が使用される。この実装では、リンクリストの先頭（最初の要素）がスタックのトップを表す。新しい要素はリンクリストの先頭に追加され、同じ位置から削除される。

3.3.2 キュー

スタックとは対照的に、キューは FIFO (First In First Out) 型のデータ構造であり、最初に追加された要素が優先され、最初に取り出される。スタックの実装と同様に、キューも単方向リンクリストを用いて実装される。キューの構造体はコード 4 に示されている。

コード 4: キューの構造

```
struct QueueNode{
    Node *graph_node; // グラフノードへのポインタ
    TreeNode *tree_parent; // 全域木ノードへのポインタ
    QueueNode *next;
};
struct Queue{
    QueueNode *head;
    QueueNode *tail;
    int len;
};
```

3.4 課題 1 と課題 2: DFS と BFS の実装

先に章 2.1.1 および章 2.1.2 で述べたように、実装した両方の探索アルゴリズムはノードをスタックまたはキューに入れる前に訪問済みとしてマークするように変更している。この修正によって、サイクルを含むグラフにおいてエンキューされたノードやプッシュされたノードが再訪されることを防げる。両アルゴリズムの実装は再帰ではなく反復を用いており、大規模なグラフにおけるスタックオーバーフローの可能性を回避している。

3.4.1 DFS の実装

DFS は章 3.3 で述べたスタックのデータ構造を用いて実装する。DFS は幅よりも深さを優先的に探索するため、アルゴリズムは以下のように動作する:

1. 空のスタックを生成する。根ノードを訪問済みにマークし、それをスタックにプッシュする。

コード 5: 全域木とスタックの初期化

```
TreeNode *root_tree = create_tree_node(
    root_node->id,
    NULL
```

```

    );
    Stack *stack = init();
    root_node->visited = true;
    push(stack, root_node, root_tree);

```

2. スタックが空でない限り, 以下を繰り返す:

- (a) スタックからノードを1つポップする. このノードが現在のノードとなる.

コード 6: スタックからノード取り出す

```

StackNode *current_stack_node = pop(stack);

```

- (b) 現在のノードの未訪問の子ノードごとに, それを訪問済みにマークし, 対応する木ノードを生成して現在の木ノードに接続し, スタックにプッシュする.

コード 7: 本ノードの各子ノードをスタックに入れる

```

Node *children = current_node->connected_nodes[i];
if (children->visited == false){
    children->visited = true;
    TreeNode *child_tree_node = create_tree_node(
                                                children->id,
                                                current_tree_node
    );
    add_child(current_tree_node, child_tree_node);
    push(stack, children, child_tree_node);
}

```

DFS 探索によって得られた全域木は返され, さらに解析を行うことができる.

3.4.2 BFS の実装

DFS の実装とは対照的に, BFS は 3.3 節で述べたキューのデータ構造を用いる. BFS は深さよりも幅を優先するため, アルゴリズムは以下のように動作する:

1. 根の全域木を初期化し, 空のキューを生成する. 根ノードを訪問済みにマークし, それをキューにエンキューする.

コード 8: 全域木とキューの初期化

```

TreeNode *root_tree = create_tree_node(

```

```

                                root_node->id ,
                                NULL
                                );
Queue *q = init_queue ();
root_node->visited = true;
enqueue(q, root_node, root_tree);

```

2. キューが空でない限り, 以下を繰り返す:

- (a) キューからノードを1つデキューする. このノードが現在のノードとなる.

```

QueueNode *current_queue_node = dequeue(q);

```

- (b) 現在のノードの未訪問の子ノードごとに, それを訪問済みにマークし, 対応する木ノードを生成して現在の木ノードに接続し, キューにエンキューする.

コード 9: 本ノードの各子ノードをキューに入れる

```

child_graph->visited = true;
TreeNode *child_tree = create_tree_node(
                                child_graph->id ,
                                current_tree_node
                                );
add_child(current_tree_node, child_tree);
enqueue(q, child_graph, child_tree);

```

BFS 探索によって得られた全域木は返され, さらに解析を行うことができる.

3.5 課題3: 連結成分数

グラフにおける連結成分数の計算方法は, 2.1 節で述べた DFS または BFS アルゴリズムを用いることができる. 本実験で実装した走査アルゴリズムは, 単一の成分に含まれるノードを訪問済みとしてマークする. 連結成分数を数えるアルゴリズムの実装をコード 10 に示す.

コード 10: 連結成分数の計算

```

reset_nodes(node_list, node_count);
int connected_nodes_count = 0;
for (int i = 0; i < node_count; i++){
    if (node_list[i].visited == false){

```

```

    TreeNode *tree_node = build_spanning_tree_bfs(
        node_list
    );
    free_tree(tree_node);
    connected_nodes_count++;
}
}
return connected_nodes_count;

```

`node_list` はグラフ内の全ノードを ID 順に格納した配列である。配列内の各ノードは、接続されたノードへのポインタを保持する。この配列を順に走査し、未訪問のノードを確認する。各未訪問ノードに対して BFS (または DFS) を実行する。走査アルゴリズムはノードを訪問済みにマークするため、同じ連結成分に含まれる全ノードは訪問済みとなる。

走査を 1 回行うごとに連結成分数を 1 増加させる。この処理は、グラフ内のすべてのノードが訪問されるまで繰り返される。

3.6 課題 4: 最大クリーク

3.6.1 最大クリーク探索には

4 実験結果

5 分析と理論

6 まとめ

7 発表についての感想

8 付録

本実験に利用したソースコードなどは GitHub にホストされている。 <https://github.com/SorataBaka/kosen-5/tree/main/graph-network>

参考文献

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

- [2] Reinhard Diestel. *Graph Theory*. Graduate Texts in Mathematics. Springer, 5 edition, 2017.
- [3] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.
- [4] Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. *Data Structures and Algorithms in C++*. Wiley, 2 edition, 2016.
- [5] Sergey Kalinichenko. non-recursive dfs (when to mark it visited?). <https://stackoverflow.com/questions/19757342/non-recursive-dfs-when-to-mark-it-visited>, November 2013. Answer posted on Stack Overflow.
- [6] Edward F. Moore. The shortest path through a maze. In *Proceedings of the International Symposium on the Theory of Switching*, pages 285–292, Cambridge, MA, USA, 1959. Harvard University Press.
- [7] Robert Endre Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [8] Trusted Analytics. k-clique percolation — trusted analytics python api documentation. https://pythonhosted.org/trustedanalytics/python_api/graphs/graph-/kclique_percolation.html. Accessed: 2025-09-07.
- [9] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, 2 edition, 2001.